Lab 6 - Ultrasonic Distance Detection



by

Yuwei Zhao[1], Haohao Qi[2], Yilin Zhang[3]

In contributions of

[1] *Data Processing & Experiment Implementation*
[2] *Initial Draft & Images Collection*
[3] *Paper Writing & Experiment Implementation*

ROBOTICS

SCHOOL OF MATHEMATICAL AND COMPUTER SCIENCES

HERIOT-WATT UNIVERSITY

May 2025

1 Introduction

Experiment: Measurement of the maximum distance that can be measured to an obstacle at different angles
This laboratory aims on discovering the maximum distance that can be measured in a series of particular angles. A radar chart of all accessible data is produced to offer a visible approach. We measured the maximum distance of detectable obstacles at different angles within a range of 0 to 400 centimeters from the obstacle and plotted the data into a radar chart to intuitively display the detection range in various directions. We find that there are much more figure from 80 to 100 degrees.

2 Theory and Technical Background

To achieve ultrasonic distance measurement, the main components we use are the **HC-SR04** ultrasonic distance measurement module and the **SG90** servo motor.

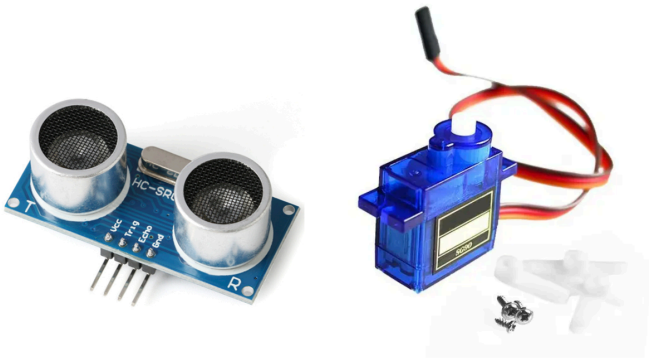


Figure 1: HC-SR04 ultrasonic distance measurement module

2.1 HC-SR04 ultrasonic distance measurement module

The **HC-SR04** (Figure **??**) ultrasonic distance measurement module consists of an ultrasonic transmitter and a receiver. There are two important pins. **Trig pin**: In fact, it is trigger, which means to trigger or cause. At that time, only a high level of 10 s needs to be applied to this pin, and HC-SR04 will automatically emit 8 square waves of 40KHz (that is, ultrasonic waves). **Echo pin**: echo actually means echo. When the HC-SR04 successfully emits ultrasonic waves outward, this pin will become a high level, and the high level will continue until the HC-SR04 receives the echo.
The transmitter sends ultrasonic waves in a specific direction.

- When the ultrasonic waves encounter an obstacle, they are reflected back, and the receiver captures the reflected waves. By calculating the time difference between the emission and reception of the ultrasonic waves, combined with the speed of sound, the distance can be determined.
- When the ultrasonic wave is emitted, the Echo pin becomes high.

- When the ultrasonic wave is received by the receiver, the Echo pin becomes low.

By measuring the duration of the high state of the Echo pin, the time of ultrasonic wave propagation can be obtained.

2.2 SG90 servo motor

The SG90 (Figure **??**) servo motor is a position (angle) servo driver, suitable for control systems that require continuous angle changes and the ability to maintain a specific angle. It is mainly composed of a casing, circuit board, DC motor, gear reduction set, and position detection element (control circuit), forming an automatic (closed-loop) control device.
The data cable of the servo should be connected to the **PWM** pin, thus, the PWM value can be correctly passed in.

3 Method

Abridge	Full Name
IDE	Integrated Development Environment
PWM	Pulse-Width Modulation
Trig	Trigger

Keywords: "distance-angle" radar chart, ADC, control variable, Ultrasonic distance measurement
Experient: Measurement of the maximum distance that can be measured to an obstacle at different angles
To achieve the function, we have carefully designed the following circuit diagram (3):

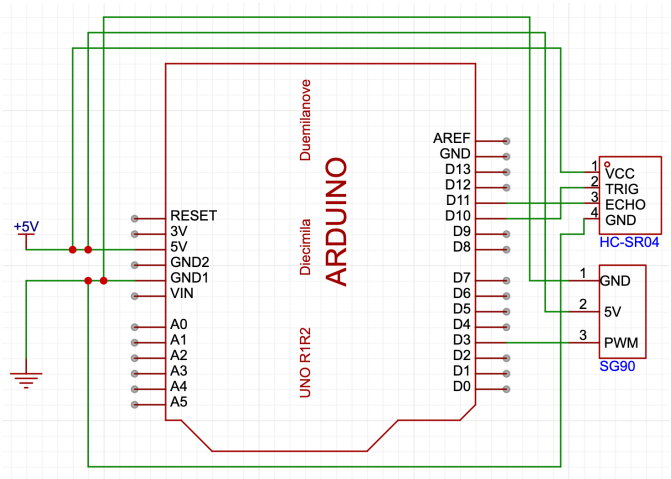


Figure 2: circuit diagram

In accordance with the designed circuit diagram, the actual wiring figures (4) of our vehicle (front and above) are as follows:

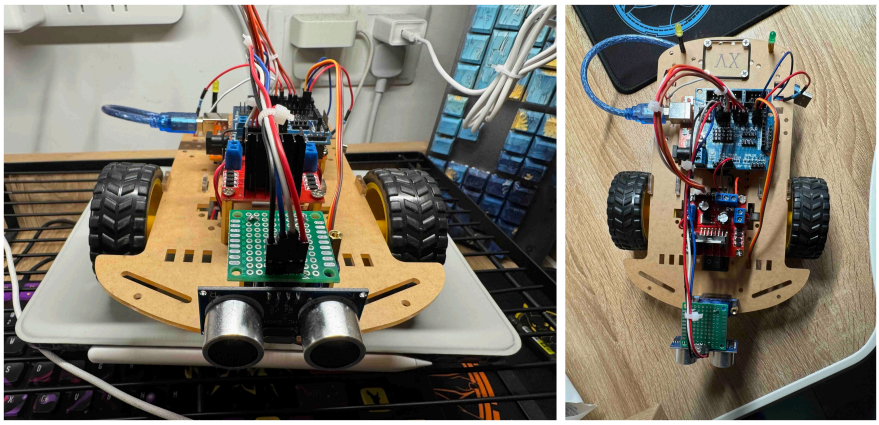


Figure 3: actual wiring of vehicle

We divide the laboratory into 3 steps and the workflow figure (**??**) is as follows:

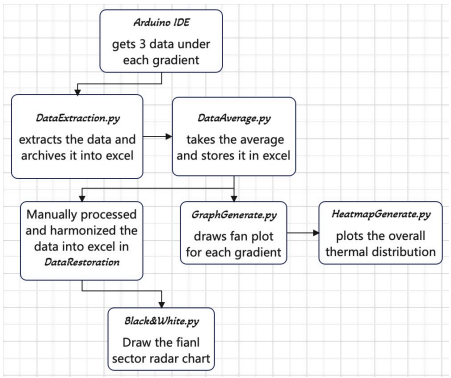


Figure 4: WorkFlow

3.1 Data Acquisition

Raw data is collected via Arduino IDE with triplicate measurements per group (B.6). After selecting the obstacle, measurements are taken at intervals of 40 centimeters within a range of 0 to 400 centimeters from the obstacle. The measurement angles span from 0 to 180 degrees, with the direction directly in front of the vehicle's nose defined as 90 degrees. The formulas (1) used are as follows:

$$Distance = Velocity \times Time \tag{1}$$

In the experimental environment, where the air is dry and the temperature is 20 degrees Celsius, the speed of sound is approximately 343 meters per second, which is equivalent to 34,300 centimeters per second. Since the ultrasonic wave travels twice the distance from the transmitter to the target and back to the receiver, the actual distance measurement is related to the time difference as follows: *a distance of 1 centimeter corresponds to a time difference of 58.3 microseconds*, which can be approximated to 58 microseconds for simplified calculations.

3.2 Extraction and Production

Several Python scripts are used in this step.
Firstly, We design a simple Python program *DataExtraction.py* (B.1). The Python code is based on *pandas*, and it extracts those raw data and automatedly store them into an *Excel* file. Additionally, there are three pieces of values. We make another Python script *DataAverage.py* (B.2) to calculate the average value.

3.3 Radar Chart Drawing

Finally, through *Black&White.py* (B.3), a monochrome radar chart is drawn as required. A *black* and *white* design are used to offer a better a view experience.

4 Results

We measured three sets of data every 40cm within the range of 0 to 400cm, calculated the average of the three sets of data (B.4), and plotted them into an average radar chart (Figure 5).

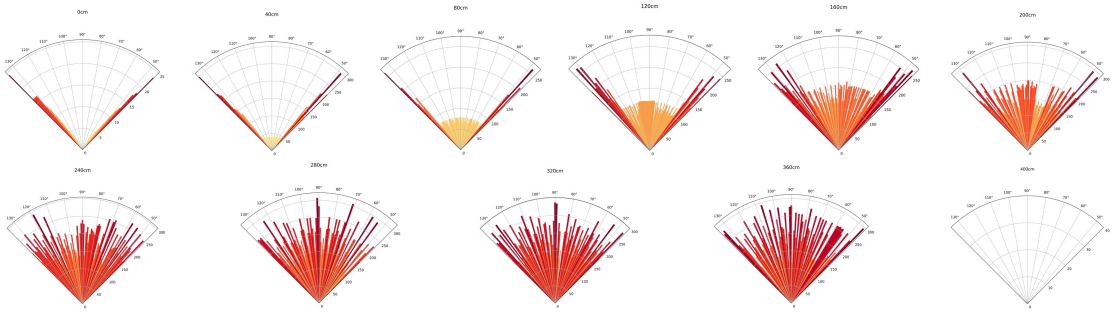


Figure 5: A “distance-angle” radar chart divided every 40cm within the range of 0 to 400cm

By integrating the data from angles ranging from 0 to 180 degrees and distances ranging from 0 to 400cm, we have created the following radar chart (Figure 6) using *Heatmap-Generate.py* (B.4). It illustrates how far obstacles can be detected at different angles when the forward linear distance is the same.

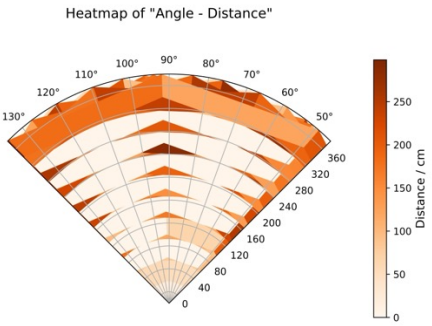


Figure 6: A distribution heatmap divided every 40cm within the range of 0 to 400cm

Taking all the data into account, we have integrated the data from 0 to 180 degrees and 0 to 400cm, and created the following radar chart (Figure 7) using *Black&White.py* (B.3) to illustrate the maximum distance at which obstacles can be detected at each angle.

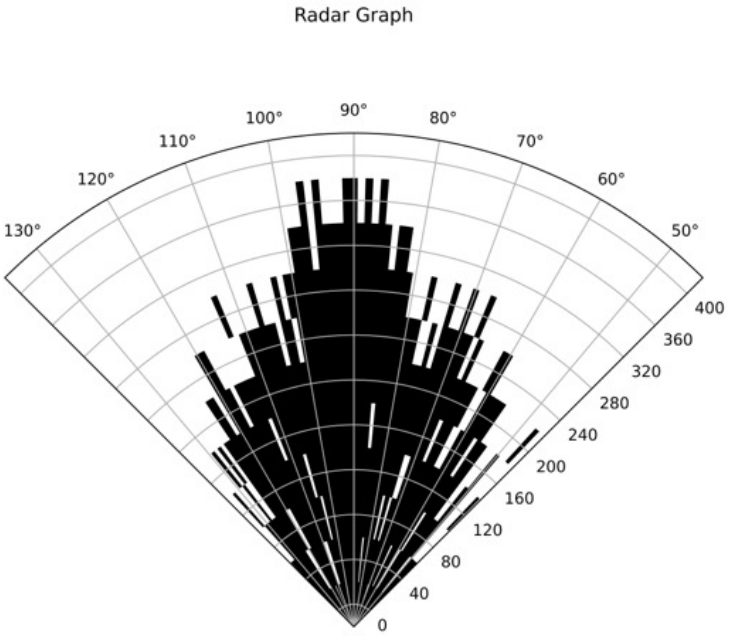


Figure 7: "Distance-Angle" Sector Radar Chart

5 Conclusions

5.1 Conclusion and Summary

In this experiment, we accomplished the following tasks:

1. We collected three sets of data at angles ranging from 0 to 180 degrees and distances ranging from 0 to 400 cm from the obstacle, and then calculated the average of these three sets of data.

2. Based on the calculated averages, we created two radar charts. One illustrates the maximum distance at which obstacles can be detected at each angle, while the other shows how far obstacles can be detected at different angles when the forward linear distance is the same.

5.2 Limitations of the experiment

1. We cannot rule out all environmental influences, as the ultrasonic waves may have reflected off objects other than the intended obstacles.

2. After obtaining the experimental data, we manually removed outliers, which may have led to the inadvertent deletion of some valid values.

A Contribution Statement

The contributions of each team member are as follows:
- Yuwei Zhao: Responsible for data processing and experiment implementation.
- Yilin Zhang: Contributed to the initial draft and collected supporting images.
- Haohao Qi: Wrote the final report and participated in experiment implementation.

B Source Code

B.1 DataExtraction.py

```
import pandas as pd

data_text = """
Angle: 135°; Distance: 218.58 cm
Angle: 134°; Distance: 274.31 cm
Angle: 133°; Distance: 0.00 cm
Angle: 132°; Distance: 0.00 cm
Angle: 131°; Distance: 188.98 cm
... (from Arduino IDE)
Angle: 49°; Distance: 115.88 cm
Angle: 48°; Distance: 197.79 cm
Angle: 47°; Distance: 275.62 cm
Angle: 46°; Distance: 0.00 cm
Angle: 45°; Distance: 0.00 cm
"""

data_lines = data_text.strip().split("\n")
extracted_data = []

for line in data_lines:
    parts = line.split(";")
    angle_part = parts[0].strip()
    distance_part = parts[1].strip()

    angle = int(angle_part.split(":")[1].strip().replace("°", ""))
    distance = float(distance_part.split(":")[1].strip().replace("cm",
        "").strip())

    extracted_data.append({"Angle": angle, "Distance / cm": distance})

df = pd.DataFrame(extracted_data)

df.to_excel("./DataProcessing/360cm/360cm_data_03.xlsx", index=False)
```

B.2 DataAverage.py

```
import pandas as pd

```

```
file_paths = [
    "./DataProcessing/280cm/280cm_data_01.xlsx",
    "./DataProcessing/280cm/280cm_data_02.xlsx",
    "./DataProcessing/280cm/280cm_data_03.xlsx"
]

data_frames = [pd.read_excel(file) for file in file_paths]

combined_data = pd.concat(data_frames)
average_data = combined_data.groupby(combined_data.index).mean()

average_data = average_data.round(2)

output_file = "./DataAverage/280cm_average_data.xlsx"
average_data.to_excel(output_file, index=False)
```

B.3 Black&White.py

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

file_paths = [f"./DataRestoration/{i}cm_0&1_data.xlsx" for i in range
    (0, 401, 40)]

distance_steps = np.arange(0, 401, 40)
angle_steps = np.arange(45, 136)
data_matrix = np.zeros((len(distance_steps), len(angle_steps)))

for idx, file_path in enumerate(file_paths):
    df = pd.read_excel(file_path)
    for _, row in df.iterrows():
        angle = int(row["Angle"])
        distance = row["Distance / cm"]
        if 45 <= angle <= 135:
            angle_idx = angle - 45
            data_matrix[idx, angle_idx] = distance

binary_matrix = np.where(data_matrix > 0, 1, 0)

angles = np.deg2rad(angle_steps)
distances = distance_steps
theta, r = np.meshgrid(angles, distances)

fig, ax = plt.subplots(subplot_kw={'projection': 'polar'}, figsize
    =(8, 8))

cmap = plt.cm.binary

heatmap = ax.pcolormesh(theta, r, binary_matrix, cmap=cmap, shading='
    auto')

```

```
ax.set_title("Radar Graph")

ax.set_thetamin(45)
ax.set_thetamax(135)
ax.set_rticks(distance_steps)
ax.set_rlabel_position(90)

plt.savefig("./Photos/RadarGraph.png", dpi=1200)
plt.show()
```

B.4 GraphGenerate.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

file_name = "./DataAverage/280cm_average_data.xlsx"

df = pd.read_excel(file_name)
angles = df["Angle"].values
distances = df["Distance / cm"].values

valid_indices = distances > 0
angles = angles[valid_indices]
distances = distances[valid_indices]

theta = np.deg2rad(angles)

fig, ax = plt.subplots(subplot_kw={'projection': 'polar'}, figsize
    =(8, 8))

cmap = plt.cm.YlOrRd
vmin = 0
vmax = max(distances)
norm = plt.Normalize(vmin=vmin, vmax=vmax)
colors = cmap(norm(distances))

for i in range(len(theta)):
    ax.bar(
        theta[i],
        distances[i],
        width=np.deg2rad(1),
        color=colors[i],
        edgecolor="none",
        alpha=1.0
    )

sm = plt.cm.ScalarMappable(cmap=cmap, norm=norm)
sm.set_array([])

ax.set_title("280cm", va="bottom", fontsize=14)
ax.set_thetamin(45)
```

```
ax.set_thetamax(135)
ax.set_theta_zero_location("E")

plt.savefig("./Photos/angle-distance/280cm.png", dpi=1200)
plt.show()
```

B.5 HeatmapGenerate.py

```
import pandas as pd
import mlx.core as mx
import matplotlib.pyplot as plt

file_paths = [f"./DataAverage/{i}cm_average_data.xlsx" for i in range
    (0, 401, 40)]

distance_steps = mx.arange(0, 400, 40)
angle_steps = mx.arange(45, 136)
data_matrix = mx.zeros((len(distance_steps), len(angle_steps)))

for idx, file_path in enumerate(file_paths):
   df = pd.read_excel(file_path)
   for _, row in df.iterrows():
      angle = int(row["Angle"])
      distance = row["Distance / cm"]
      if 45 <= angle <= 135:
         angle_idx = angle - 45
         data_matrix[idx, angle_idx] = distance

angles = mx.degrees(angle_steps)
distances = distance_steps
theta, r = mx.meshgrid(angles, distances)

fig, ax = plt.subplots(subplot_kw={'projection': 'polar'}, figsize
    =(8, 8))
cmap = plt.cm.Oranges
heatmap = ax.pcolor(theta, r, data_matrix, cmap=cmap, shading='auto')

cbar = plt.colorbar(
   heatmap,
   ax=ax,
   orientation='vertical',
   pad=0.1,
   shrink=0.6
)
cbar.set_label("Distance / cm", fontsize=12)

ax.set_title("Heatmap of \"Angle - Distance\"", va="bottom", fontsize
    =14)
ax.set_thetamin(45)
ax.set_thetamax(135)
ax.set_rticks(distance_steps)
ax.set_rlabel_position(90)
```

```
42
43 plt.savefig("./Photos/heatmap.png", dpi=1200)
44 plt.show()
```

B.6 distanceDetect.ino

```
1  #include <Servo.h>
2  Servo engine;
3
4  // steering gear
5  int enginePin = 3;
6
7  // ultrasonic sensor
8  int ultrasound_TrigPin = 10;
9  int ultrasound_EchoPin = 11;
10
11 int counter = 0;
12
13 void setup() {
14   // put your setup code here, to run once:
15   Serial.begin(9600);
16   // Engine Driver
17   engine.attach(enginePin);
18   // Ultrasound Driver
19   pinMode(ultrasound_TrigPin, OUTPUT);
20   pinMode(ultrasound_EchoPin, INPUT);
21 }
22
23 void loop() {
24   // put your main code here, to run repeatedly:
25   if(counter < 6) {
26     engineTurn();
27   } else if(engine.attached()) {
28     engine.detach();
29     Serial.println("STOP!!!");
30   }
31 }
32
33 int angle = 90;
34 int step = 1;
35 unsigned long previousMillis = 0;
36
37 void engineTurn() {
38   unsigned long currentMillis = millis();
39
40   if(currentMillis - previousMillis >= 20) {
41     previousMillis = currentMillis;
42
43     engine.write(angle);
44
45     ultrasoundControl();
46
```

```
    angle += step;

    // Change the direction when arrive the boundary
    if(angle >= 135) {
      angle = 135;
      step = -step;
      counter++;
    } else if(angle <= 45) {
      angle = 45;
      step = -step;
      counter++;
    }
  }
}

float duration, distance;

void ultrasoundControl() {
  digitalWrite(ultrasound_TrigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(ultrasound_TrigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(ultrasound_TrigPin, LOW);

  duration = pulseIn(ultrasound_EchoPin, HIGH, 25000);
  distance = (duration * 0.0343) / 2;

  Serial.print("Angle: ");
  Serial.print(angle);
  Serial.print("°; Distance: ");
  Serial.print(distance);
  Serial.println(" cm");
}
```